Table of Contents

System setup: 2
Project files: 2
Key configuration constants: 2
Local setup steps: 2
Deployment steps on Streamlit Community Cloud: 3
GitHub link: 3
Deployment link: 3
Deployment steps: 3
Architecture and Flow diagram: 4
Agent roles: 6
Highlevel overview: 6
Detailed overview: 7
Technical Limitations: 8
Development challenges: 8

System setup:

Project files:

- App_query_agenticAI.py -> Main application — all logic and UI
- .env -> Local secrets — never committed to Git
- requirements.txt -> Pinned dependencies for deployment
- .gitignore -> Excludes .env and .venv from Git
- README.md -> Setup instructions for GitHub
- .venv/ -> Local virtual environment — not committed

Key configuration constants:

- GEMINI_MODEL - gemini-2.5-flash
- EMBED_MODEL - gemini-embedding-001
- CHUNK_SIZE - 512 chars
- CHUNK_OVERLAP - 100 chars
- TOP_K - 5 chunks
- COLLECTION_NAME - enterprise_docs

Local setup steps:

1. Create virtual environment — python -m venv .venv
2. Activate venv — PowerShell: Set-ExecutionPolicy RemoteSigned -Scope Process then .venv\Scripts\Activate.ps1
3. Install dependencies — pip install -r requirements.txt
4. Create .env file — add GOOGLE_API_KEY={your_key}
5. Run the app — streamlit run app.py

Deployment steps on Streamlit Community Cloud:

GitHub link:

- https://github.com/vktraja-ops/documents-qa

Deployment link:

- https://documents-app-abnab74vdoddv2yjhevwfn.streamlit.app/

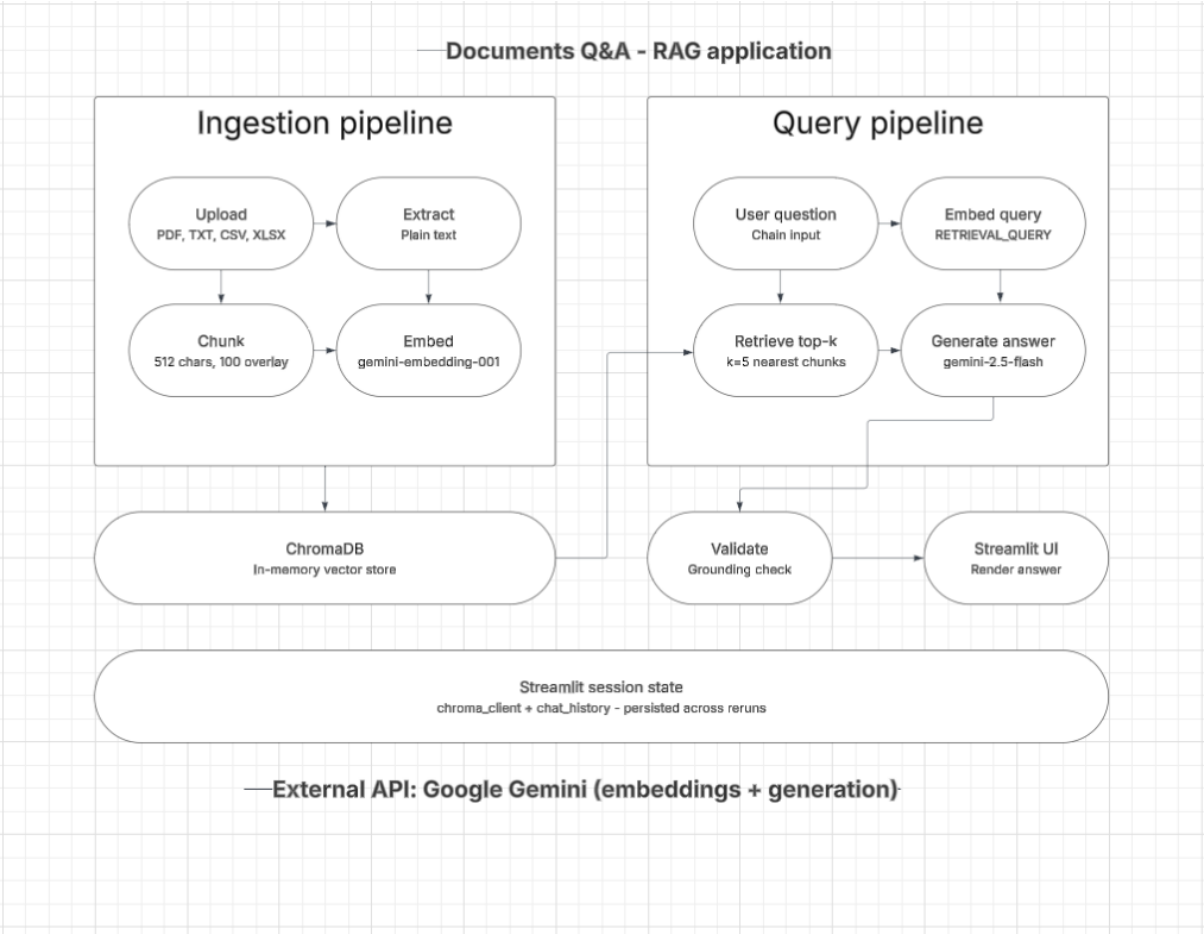
Deployment steps:

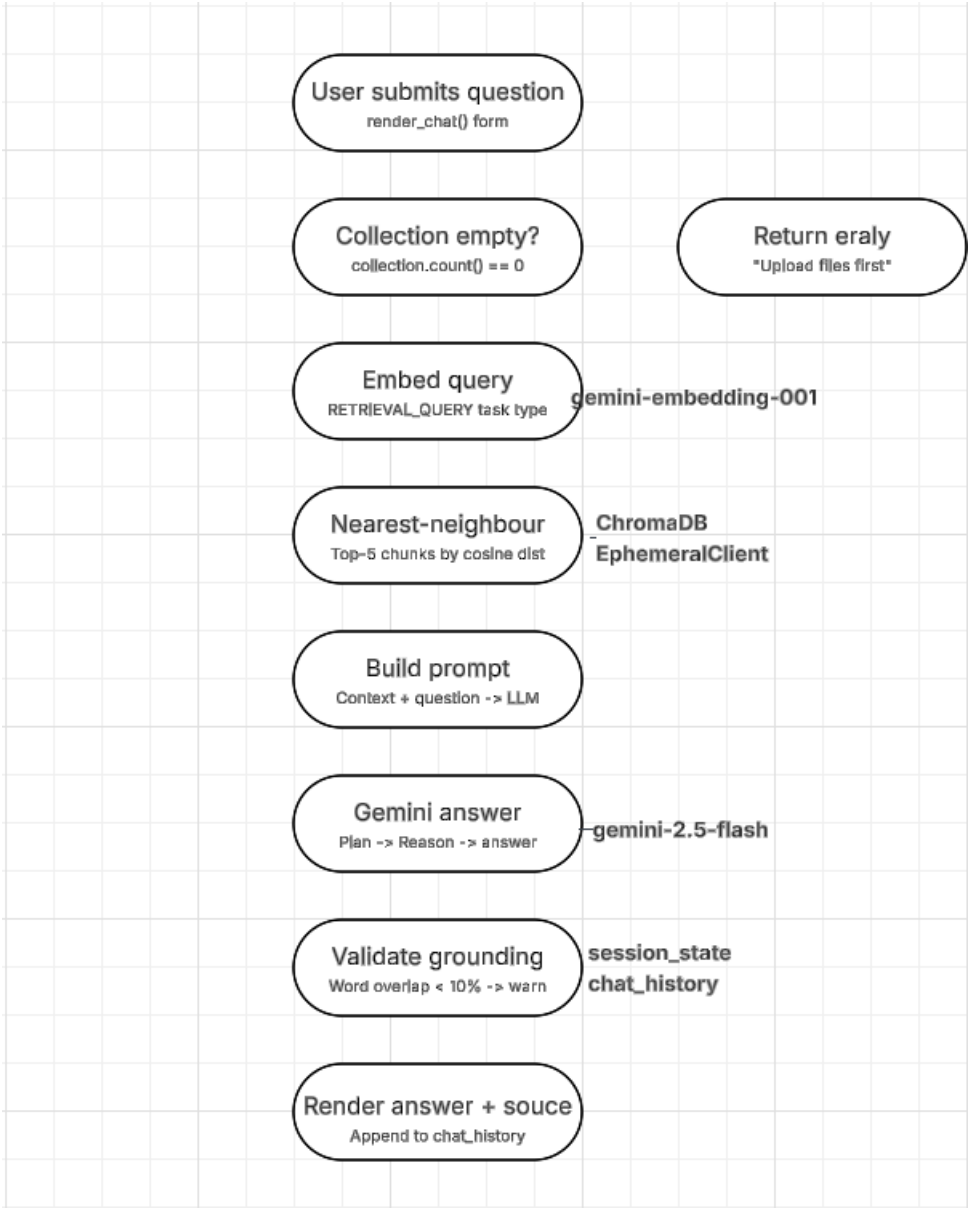
1. Go to https://share.streamlit.io/ and sign in with your GitHub account
2. Click -> **Create app**
3. Fill in:
    - **Repository** -> `vktraja-ops/documents-qa`
    - **Branch** -> `main`
    - **Main file path** -> `app_query_agenticAI.py`
4. Click -> **Advanced settings** and add your secret: *GOOGLE_API_KEY = "google_api_key"*

    This replaces your `.env` file in production.

5. Click -> **Deploy**

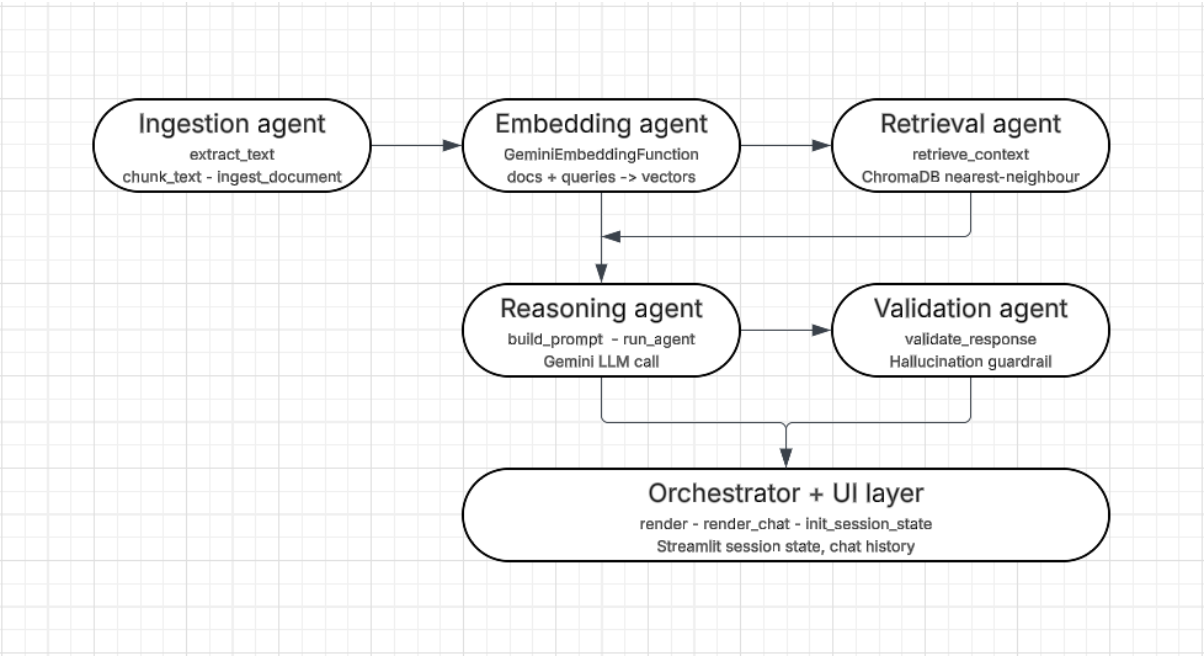
Architecture and Flow diagram:



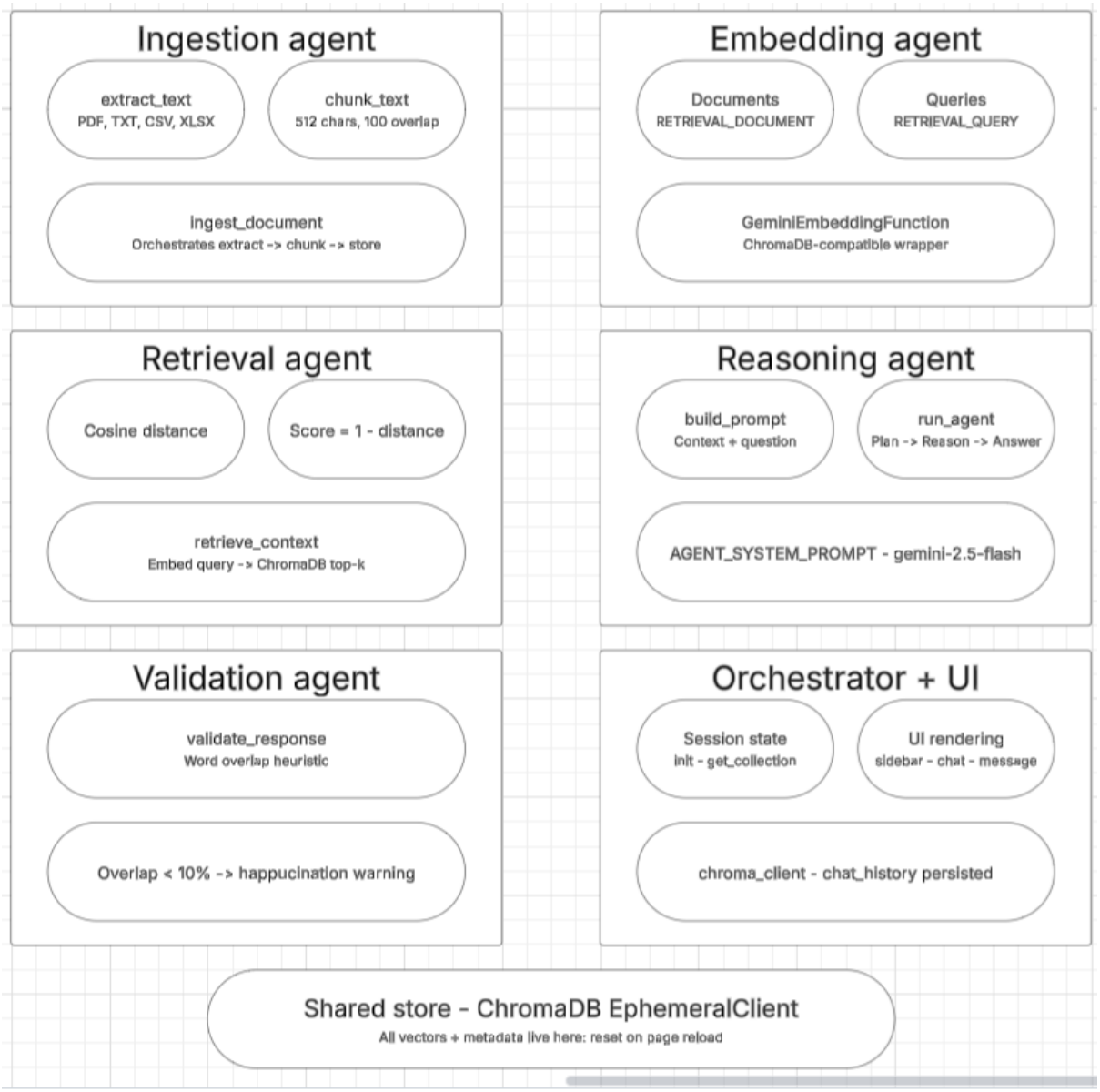

User submits question
render_chat() form
Collection empty?
collection.count() == 0
Return eraly
"Upload files first"
Embed query
RETRIEVAL_QUERY task type
gemini-embedding-001
Nearest-neighbour
Top-5 chunks by cosine dist
ChromaDB
EphemeralClient
Build prompt
Context + question -> LLM
Gemini answer
Plan -> Reason -> answer
gemini-2.5-flash
Validate grounding
Word overlap < 10% -> warn
session_state
chat_history
Render answer + souce
Append to chat_history

Agent roles:

Highlevel overview:



Detailed overview:



Technical Limitations:

- **In-memory vector store:** ChromaDB EphemeralClient holds all embeddings in RAM. Every page refresh or app restart wipes the entire knowledge base — users must re-upload all documents.
- **Duplicate chunk IDs on re-upload:** Chunk IDs are derived from the filename. Re-uploading the same file a second time silently overwrites old chunks — but uploading a slightly renamed copy creates duplicates that pollute retrieval results.
- **Hallucination detection is a heuristic:** The vocabulary-overlap check (words ≥ 5 chars) produces false positives when the model correctly paraphrases, and false negatives when it copies irrelevant context words. The 10% threshold is arbitrary.
- **Character-level chunking loses structure:** Splitting by character count (512 chars) ignores document structure — a chunk may split mid-sentence, mid-table-row, or mid-paragraph. This degrades retrieval quality for structured67 formats like CSV and XLSX.
- **Answers appear all-at-once:** The Gemini call blocks until the full response is ready. For long answers this means a multi-second blank wait before any text appears, which feels unresponsive compared to streaming token-by-token output.

Development challenges:

- **Library version churn:** Three APIs were removed or renamed during development: PyPDF2 → pypdf, chromadb.Client(Settings(...)) → EphemeralClient(), and st.experimental_rerun() → st.rerun().
    - **Resolved**: Pinned versions in requirements.txt
- **Embedding values not plain lists:** e.values on a Gemini embedding returns a protobuf repeated field, not a Python list[float]. ChromaDB silently rejected it until explicitly wrapped with list().
    - **Resolved**: Wrapped all e.values accesses with list() and added None guards.
- **Streamlit rerun model complexity:** Streamlit reruns the entire script on every interaction. Without careful initialisation guards in init_session_state(), the ChromaDB client and chat history were being recreated on every user action, wiping state.
    - **Resolved**: if "key" not in st.session_state guards prevent re-initialisation on reruns.
- **PowerShell execution policy:** Windows blocked .venv\Scripts\Activate.ps1 by default due to RemoteSigned execution policy restrictions, preventing virtual environment activation during development setup.
    - **Resolved**: Used Set-ExecutionPolicy RemoteSigned
- **Type annotation on subscript:** Python 3.14 stricter parsing rejected st.session_state["key"]: list[dict]
    - **Resolved**: Converted to st.session_state["key"] = [] # list[dict].